xCurl 1.1.0

Manuel pratique d’intégration, de flux et de contrats

Pas à pas, sans supposer une connaissance interne de la bibliothèque

Champ	Valeur
Version	1.1.0 stable
PHP pris en charge	^8.5.12
Date des métadonnées de release	25/10/2026
Distribution	Composer/Packagist recommandé ; alternatives GitHub VCS et standalone
Dépôt source	https://github.com/absolutoffice/xcurl
Document	xCurl-1.1.0-Manuel-pratique-fr.pdf
Base de vérification	Code 1.1.0, métadonnées et régressions incluses examinés le 25/10/2026

À qui s’adresse ce manuel

Aux utilisateurs qui doivent installer, intégrer, exploiter ou diagnostiquer xCurl sans déduire son comportement de toute l’implémentation. Les anciennes affirmations QA destinées aux mainteneurs ne sont pas présentées comme preuve actuelle de la release.

Absolut Office - documentation utilisateur

0. Si vous ne lisez que cinq minutes

Le modèle complet de xCurl sur une seule page.

Règle mentale principale

xCurl possède deux couches. Un TRANSPORT exécute exactement une requête HTTPS. HttpClient décide des redirects, retries, Retry-After et de la deadline globale. HttpTransportFactory ne fait que sélectionner un backend selon les capacités disponibles au runtime.

Question	Réponse courte
Comment l'installer ?	Composer via Packagist : composer require absolutoffice/xcurl:^1.1.
Quelle API utiliser normalement ?	HttpClient::send(HttpRequest, RequestOptions).
Comment le backend est-il choisi ?	HttpTransportFactory::create() ou createStrict().
Y a-t-il un failover après l'échec d'une requête ?	Non. Le fallback n'a lieu que lors de la sélection selon les capacités du runtime.
Qu'est-ce qui est retenté par défaut ?	GET/HEAD ; erreurs transitoires, 408, 429 et 5xx ; au plus deux retries.
Qui suit les redirects ?	Uniquement HttpClient. Le transport renvoie le 3xx sans le suivre.
Body null et body vide sont-ils identiques ?	Non. null = pas d'entité ; chaîne vide = entité explicite de 0 octet.
Comment télécharger vers un fichier ?	RequestOptions(targetPath: ...). La cible ne doit pas exister.
Comment diagnostiquer le backend ?	HttpTransportFactory::diagnostics().
Comment vérifier le runtime installé ?	composer check-platform-reqs --no-dev, diagnostics() puis une requête HTTPS simple.

Plan du manuel

Cas	Pourquoi
1-4	Modèle mental, installation, première requête et sélection du transport.
5-8	Contrats request/response, téléchargements et erreurs typées.
9-13	Retries, redirects, temps, TLS/limites et diagnostic.
14-16	Vérification actuelle, distribution Composer et carte de l'API publique.
17-22	Recettes, anti-patterns, dépannage, déploiement, maintenance et références.
Annexe	Aide-mémoire d'une page.

1. Modèle mental : rôle de chaque couche

Décidez d’abord si vous avez besoin d’une seule requête bas niveau ou d’une opération logique avec policy.

Concept	Ce que cela signifie dans xCurl	Ce que cela NE signifie PAS
Fallback	Sélectionner cURL, streams HTTPS ou socket selon les capacités actuelles du runtime.	Essayer un autre backend après une erreur DNS/TLS/lecture.
Retry	Répéter une requête logique selon RetryPolicy avec le même transport sélectionné.	Changer automatiquement de transport.
Redirect	HttpClient suit Location et applique les règles méthode/body/identifiants.	Le transport suit les redirects tout seul.

Phrase à retenir

Factory choisit le moteur. Transport fait une tentative. HttpClient organise l’opération.

Cas

Cas	Utiliser	Pourquoi
Une requête, sans policy retry/redirect	HttpTransportInterface::request()	Contrat réduit et prévisible.
Usage normal de l’application	HttpClient::send()	Sépare les données de requête de la policy et conserve l’historique.
Appel scalaire de compatibilité	HttpClient::request()	Wrapper compatible sur send().
Diagnostic/test déterministe du backend	HttpTransportFactory::createNamed()	Ce n’est pas le chemin normal de sélection métier.

2. Installation et runtime

Composer est la voie recommandée pour les consommateurs en 1.1.0. Le mode standalone reste disponible comme fallback de compatibilité.

Recommandé : Composer / Packagist

Installez la ligne 1.1 dans le projet consommateur. Composer applique l'exigence PHP et fournit l'autoload PSR-4.

```
composer require absolutoffice/xcurl:^1.1
```

Alternative directe via GitHub VCS

À utiliser si vous souhaitez épingler explicitement le tag stable GitHub comme dépôt VCS plutôt que la résolution Packagist normale.

```
composer config repositories.xcurl vcs https://github.com/absolutoffice/xcurl.git
composer require absolutoffice/xcurl:1.1.0 --no-plugins --no-scripts
```

Bootstrap de l'application

Chargez une seule fois l'autoloader du projet consommateur. N'exécutez pas une installation Composer séparée dans vendor/absolutoffice/xcurl.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
```

Note de production

Le serveur de production a besoin de l'arbre vendor déployé et de son autoloader généré. L'exécutable composer peut rester dans CI/build si les dépendances sont installées avant le déploiement.

Fallback standalone

Le dépôt conserve un autoloader local léger. Utilisez-le uniquement lorsque le déploiement Composer n'est pas adapté ; le loader standalone n'effectue pas les platform checks de Composer.

```
require_once '/path/to/xcurl/autoload.php';
```

3. Première requête : chemin recommandé

Utilisez HttpClient lorsque l’opération peut nécessiter redirects, retries, deadline ou historique de tentatives.

```
use AbsolutOffice\XCurl\Client\HttpClient;
use AbsolutOffice\XCurl\Client\RequestOptions;
use AbsolutOffice\XCurl\HttpRequest;
use AbsolutOffice\XCurl\HttpTransportFactory;

$client = new HttpClient(HttpTransportFactory::create());
$result = $client->send(
    new HttpRequest(
        'GET',
        'https://example.com/data.json',
        ['User-Agent: MyProject/1.0'],
    ),
    new RequestOptions(
        timeoutSeconds: 30,
        deadlineSeconds: 90,
    ),
);

$response = $result->response;
```

Ce qui se passe

1. La factory inspecte les capacités du SAPI courant et crée un seul transport.
2. HttpClient reçoit la requête immuable et les options d’exécution.
3. Le transport valide méthode, URL, headers, body, limites et target lorsque possible avant l’I/O réseau.
4. Le transport sélectionné effectue une seule requête HTTPS.
5. HttpClient décide de retourner, rediriger ou retenter.
6. Le résultat contient réponse finale, URL finale, compteurs et historique typé.

Réponse HTTP != succès métier

Un 404, 409 ou 500 reste une réponse HTTP. xCurl ne lève pas une exception uniquement parce que le status est 4xx/5xx ; la policy peut retenter certains status et l’application décide du sens métier final.

4. Sélection du transport et fallback

La sélection dépend des capacités disponibles, pas d'un échec ultérieur d'une requête.

Mode	Ordre de sélection	Quand l'utiliser
create()	cURL -> streams HTTPS -> socket TLS	Compatibilité générale ; chemin par défaut.
createStrict()	cURL -> socket TLS	Quand xCurl doit appliquer lui-même des contrôles de framing plus stricts.

Pas de failover par requête

Si create() sélectionne cURL et que la requête échoue ensuite sur DNS, timeout, TLS ou lecture, xCurl n'essaie pas silencieusement streams ou socket. HttpClient peut retenter le même transport si la policy l'autorise.

Symptôme

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

5. Contrat de request : entrée

Tous les transports fournis partagent la même validation afin d’accepter ou de rejeter une même entrée de façon cohérente.

Champ	Règle	Exemple
Méthode	Token HTTP valide ; normalisé en majuscules ; CONNECT est rejeté.	get -> GET ; CONNECT -> INVALID_REQUEST
HEAD	Un body explicite est rejeté, même vide.	HEAD + body "" -> INVALID_REQUEST
URL	HTTPS absolu uniquement ; URI ASCII RFC 3986 ; le non-ASCII doit être percent-encoded ; userinfo rejeté.	https://example.com/a%20b
Headers	Pas de contrôles dangereux ; nom token ; Host dupliqué rejeté.	Host une seule fois et autorité valide.
Transfer-Encoding	Non pris en charge dans le request.	Utiliser un framing Content-Length exact.
Content-Length	S’il est fourni, une seule fois et égal aux octets du body ; sinon xCurl le synthétise pour un body explicite.	"abc" -> 3 octets

Sémantique du body

Valeur	Signification	Content-Length
null	Pas d’entité de request.	Pas de Content-Length synthétisé ; si déclaré par l’appelant, seulement 0.
""	Entité explicite de zéro octet.	Content-Length: 0 synthétisé s’il manque.
"abc"	Entité binaire de 3 octets.	Content-Length: 3 synthétisé s’il manque.

Limite du request

maxRequestBytes est optionnel. Au-delà, échec avant le réseau avec TransportError::REQUEST_LIMIT non transitoire. Une limite inférieure à 1 octet est invalide.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

6. Contrat de response : sortie

Chaque transport fourni renvoie un HttpResponse normalisé.

Champ	Signification
statusCode	Status HTTP final, 100..599.
headers	Bloc final normalisé ; trailers possibles si exposés.
body	Body en mémoire lorsqu'aucun fichier target n'est utilisé.
bytes	Octets de body reçus ou écrits.
targetCreated	true uniquement lorsque cette tentative terminée a créé la cible de façon exclusive.

Sémantique sans body

HEAD et les status 204, 205 et 304 sont sans body. 101 Switching Protocols n'est pas pris en charge. Une réponse qui se termine uniquement par un 1xx informatif est une erreur de protocole.

Framing strict

Cas	Signification
Transfer-Encoding + Content-Length	Rejeté comme ambigu.
Transfer-Encoding autre que chunked	Rejeté.
Content-Length contradictoire	Rejeté.
Body plus court que Content-Length	Erreur READ transitoire.
Streams HTTPS PHP	Backend de compatibilité ; le wrapper peut masquer Transfer-Encoding, donc createStrict() l'exclut.

7. Téléchargements et nettoyage sûr par propriété

La version 1.1.0 rend explicite la propriété du fichier téléchargé. La cible n’est pas un emplacement à écraser.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

- Avec HttpClient, targetPath ne doit pas exister ; fichiers et symlinks existants sont refusés avant la première requête.
- Les transports fournis créent les targets de body de manière exclusive. Les réponses sans body ne créent pas de target.
- En cas d’échec du transport, le transport supprime le fichier partiel qu’il a créé.
- Avant redirect ou retry sur réponse, HttpClient ne supprime qu’un intermédiaire dont le response indique targetCreated=true.
- Si un fichier apparaît sans propriété déclarée par la tentative, la continuation s’arrête avec ClientError::TARGET_CLEANUP et le fichier n’est pas supprimé.
- Utilisez un chemin temporaire neuf par opération ; validez checksum/taille/règles métier puis promouvez/renommez dans l’application.

8. Erreurs : comprendre ce qui s’est passé

Ne basez pas la logique sur les messages d’exception. Utilisez enums, transient et historique.

Erreurs du transport

TransportError	value
DNS	dns
CONNECT	connect
TLS	tls
TIMEOUT	timeout
READ	read
WRITE	write
PROTOCOL	protocol
RESPONSE_LIMIT	response_limit
REQUEST_LIMIT	request_limit
INVALID_REQUEST	invalid_request
UNAVAILABLE	unavailable
UNKNOWN	unknown

Erreurs du client

ClientError	Signification
INVALID_REDIRECT	Destination de redirect multiple, invalide ou non HTTPS.
REDIRECT_LIMIT	Nombre maximal de redirects dépassé.
DEADLINE	Temps insuffisant pour redirect, retry ou nouvelle tentative.
TARGET_CLEANUP	Continuation sûre impossible à cause de la propriété/nettoyage d’une cible intermédiaire.
TRANSPORT	Échec transport non retenté ou conditions de retry épuisées.

transient est déterminant

TransportError classe la catégorie. HttpTransportException::$transient indique si retenter le même transport peut raisonnablement réussir. RetryPolicy vérifie aussi la méthode et le budget de retries.

```
try {
    $result = $client->send($request, $options);
} catch (AbsolutOffice\XCurl\Client\HttpClientException $e) {
    error_log('client_error=' . $e->error->value);
    foreach ($e->history as $attempt) {
        error_log($attempt->action->value);
    }
}
```

9. Retries : quand xCurl recommence

La policy par défaut est prudente afin d’éviter de dupliquer les effets de bord.

Paramètre	Défaut	Effet
maxRetries	2	Au plus deux retries après la première tentative.
backoffMilliseconds	[1000, 3000]	Délai du retry #1 et #2 ; la dernière valeur est réutilisée si nécessaire.
methods	[GET, HEAD]	Seules ces méthodes sont retentées automatiquement par défaut.
statusCodes	[408, 429]	Status explicitement retryable.
retryServerErrors	true	Retente aussi 500..599.
maxRetryAfterSeconds	60	Plafond appliqué à Retry-After valide.

POST n’est pas retenté par défaut

Si vous activez POST/PUT/PATCH dans RetryPolicy, l’idempotence relève de votre application/API. xCurl ne peut pas savoir si répéter une opération à effets de bord est sûr.

10. Redirects : règles exactes

HttpClient suit les redirects manuellement ; les transports n’activent jamais le suivi automatique.

Status	Méthode suivante	Body	Headers d’entité
301	conserve	conserve	conserve
302	conserve	conserve	conserve
303	GET sauf HEAD	supprime si méthode change	supprime Content-Length et Content-Type si méthode change
307	conserve	conserve	conserve
308	conserve	conserve	conserve

Règles de sécurité des headers

- Chaque redirect suivi supprime un Host explicite afin de régénérer l’autorité.
- Les redirects cross-origin retirent Authorization par défaut.
- Les redirects cross-origin retirent Cookie par défaut.
- Ces deux comportements sont configurables dans RedirectPolicy.
- Plusieurs Location sont invalides lorsque le suivi est activé.
- maxRedirects: 0 signifie ne pas suivre ; le 3xx est retourné tel quel.
- La continuation reste exclusivement HTTPS.

11. Timeout et deadline sont différents

timeoutSeconds protège une tentative ; deadlineSeconds limite toute l’opération HttpClient.

Concept	Ce que cela signifie dans xCurl	Effet
timeoutSeconds	Une tentative de transport.	Safety timeout avec sémantique propre au backend.
deadlineSeconds	Opération HttpClient complète.	Inclut retries, redirects et sleeps ; borne chaque nouvelle tentative.

Backend

Backend	Comportement de timeoutSeconds
cURL	Timeout total du request et plafond de connexion.
Streams HTTPS	Timeout natif d’I/O + contrôle timed_out après lecture.
Socket TLS	Connexion TCP, I/O du handshake TLS et opérations de stream suivantes.

Comportement de la deadline

Avant chaque nouvelle tentative, HttpClient calcule le temps restant. S’il reste moins d’une seconde ou si le délai plus une nouvelle tentative ne tient pas, il échoue avec ClientError::DEADLINE.

12. TLS, CA privée et limites

La vérification TLS du peer et du hostname reste active. Un bundle CA personnalisé change les ancres de confiance, pas la policy de vérification.

```
$caFile = __DIR__ . '/config/private-ca.pem';

$curl    = new CurlHttpTransport($caFile);
$streams = new StreamHttpTransport($caFile);
$socket  = new SocketHttpTransport($caFile);
```

Règle	Signification
Vérification peer	Activée.
Vérification hostname	Activée.
Chemin CA vide/NUL	Configuration invalide.
maxResponseBytes < 1	Configuration invalide.
maxRequestBytes < 1	Configuration invalide.
Response dépasse la limite	TransportError::RESPONSE_LIMIT, non transitoire ; partiel propriétaire supprimé.
Request dépasse la limite	TransportError::REQUEST_LIMIT, non transitoire ; rejet avant réseau.

```
$transport = HttpTransportFactory::create(
    maxResponseBytes: 64 * 1024 * 1024,
    maxRequestBytes: 8 * 1024 * 1024,
);
```

13. Diagnostic : capacités de chaque SAPI

CLI et Apache/FPM peuvent utiliser la même version PHP avec php.ini, extensions et capacités différents. Vérifiez diagnostics() dans le SAPI qui exécute réellement la requête.

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Symptôme	Vérifier
cURL=false	Vérifier ext-curl et les fonctions cURL requises.
streams=false	Vérifier allow_url_fopen et le wrapper https.
socket=false	Vérifier ext-openssl et les transports tls/ssl.
selectedStrict=socket avec streams=true	Normal : le mode strict exclut streams.

14. Vérification en 1.1.0

Le checkout 1.1.0 possède une surface de vérification explicite et plus réduite que l'ancien arbre de développement 1.0. Ne présentez pas les anciens rapports 10/14 gates comme preuve de la 1.1.0.

Vérification côté consommateur

Après installation, vérifiez la plateforme réelle, inspectez les capacités et exécutez une requête HTTPS simple dans le SAPI cible.

```
composer check-platform-reqs --no-dev
```

```
$diag = AbsolutOffice\XCurl\HttpTransportFactory::diagnostics();
print_r($diag);
```

Vérification du checkout

Depuis la racine du dépôt, le guide actuel documente ces checks reproductibles :

```
composer validate --strict --no-check-lock
composer test
php tests/composer-consumer.php
find src tests -name '*.php' -print0 | xargs -0 -n1 php -l
php -l autoload.php
git diff --check
```

Ce que ces checks prouvent

- composer validate vérifie les métadonnées et la cohérence du package.
- composer test exécute le runner de régression sans dépendances.
- tests/composer-consumer.php vérifie le rejet de PHP 8.5.11, l'installation simulée 8.5.12, un smoke via vendor/autoload.php et le contenu réel de l'archive Composer.
- Les checks de syntaxe couvrent les src/tests/autoload.php actuels.

Ce qu'ils NE prouvent PAS

- Ils ne remplacent pas l'ancienne suite complète d'intégration cURL/streams/TLS.
- Ils n'exécutent pas l'ancienne fire matrix ni le panneau QA navigateur ; ces outils ne sont pas inclus dans le checkout actuel.
- À eux seuls, ils ne prouvent pas une installation distante depuis GitHub ou Packagist.

15. Distribution Composer et contenu du package

La version 1.1.0 est structurée comme une bibliothèque Composer normale. La seule exigence runtime du consommateur est PHP ^8.5.12 ; les outils de contribution restent en require-dev.

- PSR-4 : AbsolutOffice\XCurl\ -> src/.
- Pas de version dans le composer.json racine ; les versions stables proviennent des tags VCS.
- Les archives Composer excluent index locaux, caches, vendor, fichiers d’environnement, PDFs et matériel de clés.
- LICENSE, COPYING, runtime et tests de régression restent distribués selon les checks actuels.
- Les manuels PDF stockés dans le dépôt sont intentionnellement exclus du package Composer par les règles archive/export actuelles.

```
"require": {
    "php": "^8.5.12"
},
"autoload": {
    "psr-4": {
        "AbsolutOffice\\XCurl\\": "src/"
    }
}
```

16. Carte de l’API publique de la ligne 1.x

La référence API 1.0 reste le baseline historique de compatibilité de la ligne 1.x. Le code 1.1.0 conserve les principaux types publics et ajoute le signal de propriété HttpResponse::targetCreated.

Type	Rôle	À retenir
HttpTransportFactory	Factory/diagnostic	create, createStrict, createNamed, diagnostics, selected*
HttpTransportInterface	Contrat de transport	request(...) -> HttpResponse
CurlHttpTransport	Transport concret	cURL
StreamHttpTransport	Transport concret	Streams HTTPS de PHP
SocketHttpTransport	Transport concret	Socket TLS direct
HttpRequest	Objet valeur	method, url, headers, body
HttpResponse	Objet valeur	statusCode, headers, body, bytes, targetCreated
HttpTransportException	Exception transport	transient + TransportError
TransportError	Enum transport	12 catégories
Client\HttpClient	Orchestrateur de policy	send() préféré ; request() wrapper
Client\RequestOptions	Options d’exécution	timeoutSeconds, targetPath, deadlineSeconds
Client\RetryPolicy	Policy de retry	methods, status, backoff, Retry-After
Client\RedirectPolicy	Policy de redirect	limite + suppression des identifiants cross-origin
Client\HttpClientResult	Résultat final	response, finalUrl, history, redirects, retries
Client\HttpClientAttempt	Enregistrement de tentative	sequence, method, url, action, status/error/delay/nextUrl
Client\HttpClientAction	Enum de l’historique	RETURNED, REDIRECTED, RETRIED, FAILED
Client\HttpClientException	Exception client	ClientError + history
Client\ClientError	Enum client	5 catégories
ClockInterface	Contrat extension/test	monotonicSeconds, unixTime
SleeperInterface	Contrat extension/test	sleepMilliseconds

Surface interne

Internal*, RequestValidator, SystemClock/SystemSleeper et les helpers de capacité sont des détails d’implémentation/test ; les consommateurs normaux ne doivent pas bâtir leur logique métier autour d’eux.

17. Recettes copiables

Petits modèles pour les cas fréquents.

A. GET simple

Utiliser HttpClient + HttpRequest + RequestOptions et lire status/header/body.

```
require_once __DIR__ . '/vendor/autoload.php';

use AbsolutOffice\XCurl\HttpTransportFactory;

$transport = HttpTransportFactory::create();
$response = $transport->request(
    'GET',
    'https://example.com/',
    [],
    30,
);

echo $response->statusCode, PHP_EOL;
```

B. POST JSON sans retry automatique

POST n’est pas retryable par défaut ; fournir un body JSON brut et Content-Type.

```
$payload = json_encode($data, JSON_THROW_ON_ERROR);
$result = $client->send(
    new HttpRequest(
        'POST',
        'https://example.com/api/items',
        ['Content-Type: application/json'],
        $payload,
    ),
    new RequestOptions(timeoutSeconds: 30, deadlineSeconds: 60),
);
```

C. PUT binaire avec limites

Utiliser createStrict(maxResponseBytes: ..., maxRequestBytes: ...) si un framing strict est requis.

```
$transport = HttpTransportFactory::createStrict(
    maxResponseBytes: 4 * 1024 * 1024,
    maxRequestBytes: 16 * 1024 * 1024,
);
```

D. Gros téléchargement vers temporaire

Utiliser un targetPath unique ; valider puis promouvoir après succès.

```
$tmp = sys_get_temp_dir() . '/download-' . bin2hex(random_bytes(8)) . '.tmp';

$result = $client->send(
    new HttpRequest('GET', 'https://example.com/file.dat'),
    new RequestOptions(
        timeoutSeconds: 30,
        targetPath: $tmp,
        deadlineSeconds: 120,
    ),
);

// Validate size/checksum/business rules, then promote the file.
```

E. Désactiver le suivi des redirects

Construire RedirectPolicy(maxRedirects: 0).

```
$client = new HttpClient(
    HttpTransportFactory::create(),
    redirectPolicy: new RedirectPolicy(maxRedirects: 0),
);
```

F. Retenter POST uniquement si votre API est sûre

Ajouter POST à methods seulement avec une vraie garantie d’idempotence.

```
$retry = new RetryPolicy(maxRetries: 1, methods: ['GET', 'HEAD', 'POST']);
$client = new HttpClient(HttpTransportFactory::create(), $retry);
```

18. Anti-patterns

Évitez ces erreurs d’intégration.

Ne faites pas	Faites plutôt
« Si cURL échoue, j’essaie streams manuellement. »	Conservez le transport sélectionné ; HttpClient ne retente que si l’échec du même transport est transient et que la policy l’autorise.
Mettre URLs/fournisseurs/identifiants/règles métier dans xCurl.	Gardez-les dans l’application consommatrice.
Activer le suivi automatique des redirects dans le backend.	Laissez HttpClient gérer les redirects.
Envoyer Transfer-Encoding dans le request.	Envoyez un body normal ; xCurl utilise Content-Length exact.
Décider la logique depuis le texte d’exception.	Utilisez TransportError/ClientError, transient et history.
Utiliser createNamed() comme sélection normale.	Utilisez create()/createStrict(); createNamed() sert au diagnostic/test.
Réutiliser ou écraser une cible de téléchargement.	Utilisez un temporaire neuf pour chaque opération.
Supposer que timeout=30 vaut exactement 30.000 s partout.	Traitez-le comme safety timeout ; utilisez deadline pour le budget global.

19. Dépannage par symptôme

Commencez par le symptôme et diagnostics(); ne changez pas de transport au hasard.

Symptôme	Vérifier	Pourquoi
« No usable HTTPS transport »	HttpTransportFactory::diagnostics()	Vérifier cURL, allow_url_fopen/wrapper https et OpenSSL tls/ssl.
Fonctionne en CLI, échoue sur le web	Comparer diagnostics() dans les deux SAPI	Vérifier php.ini et extensions différents.
Échec TLS	TransportError::TLS + CA/hostname	Vérifier SAN, chaîne et trust bundle.
Target refusé avant la requête	Le chemin existe-t-il ou est-il un symlink ?	Utiliser un temporaire neuf.
Le nettoyage du target stoppe la suite	ClientError::TARGET_CLEANUP	Un fichier non possédé est apparu ou n’a pas pu être supprimé en sécurité.
POST n’est pas retenté	RetryPolicy methods	Défaut : GET/HEAD uniquement.
Redirect non suivi	RedirectPolicy maxRedirects + Location	0 désactive ; Location doit résoudre vers HTTPS.
Deadline avant maxRetries	deadlineSeconds + délais	Normal si une autre tentative complète ne tient plus.
Streams accepte un framing rejeté en strict	create() vs createStrict()	Normal : strict exclut streams.

20. Checklist de déploiement consommateur

Le déploiement normal de la 1.1.0 est Composer-first.

1. Requérir absolutoffice/xcurl:^1.1 dans l’application consommatrice.
2. Déployer l’arbre vendor généré par Composer.
3. Charger vendor/autoload.php une seule fois depuis le bootstrap.
4. Choisir create() ou createStrict() selon le framing requis.
5. Utiliser HttpClient::send() sauf besoin délibéré d’un seul appel transport.
6. Définir timeout et, pour les opérations multi-tentatives, deadline.
7. Définir des limites request/response si le cas l’exige.
8. Garder identifiants, endpoints, checksums, schemas et règles métier hors de xCurl.
9. Pour les téléchargements, utiliser un temporaire neuf et valider avant promotion finale.
10. Journaliser les enums d’erreur et history, pas le texte des messages.
11. Éventuellement exécuter composer check-platform-reqs --no-dev sur le runtime réel.

Note de production

Le serveur de production a besoin de l’arbre vendor déployé et de son autoloader généré. L’exécutable composer peut rester dans CI/build si les dépendances sont installées avant le déploiement.

21. Compatibilité et maintenance

xCurl 1.1.0 exige PHP ^8.5.12. La référence 1.0 reste le baseline documenté de la ligne 1.x, tandis que 1.1.0 ajoute des changements compatibles de distribution et de propriété sûre des téléchargements.

Changement	Traitement
Bugfix interne sans rupture publique	Peut entrer en patch/minor après les checks actuels et revue de compatibilité.
Ajout public backward-compatible	Une minor convient lorsque l’API/comportement s’étend de façon compatible.
Changement qui casse signatures/sémantique publique	Nécessite une décision explicite de versionnement sémantique.
Changement de helper interne/test	Pas un contrat consommateur, mais les régressions doivent passer.
Références QA historiques	Les conserver comme archive ; ne pas les utiliser comme preuve actuelle de 1.1.0.

22. Quelle source fait foi en cas de doute

Ce manuel explique. Le code actuel et les métadonnées de release/package définissent le comportement du checkout 1.1.0.

Source	Utiliser
src/	Implémentation runtime exacte.
composer.json	Nom, PHP, PSR-4, tooling dev et règles d’archive.
README.md	Résumé opérationnel compact.
docs/INSTALLATION.md	Chemins d’installation et commandes actuelles.
docs/releases/1.1.0.md	Changements et limites de validation de la release.
tests/regression.php	Régressions incluses sans dépendances.
tests/composer-consumer.php	Checks consommateur Composer et archive.
docs/reference/1.0.0/API-1.0.0.md	Référence historique API 1.0 pour la ligne 1.x.

Conclusion

Gardez les couches séparées : l’application décide du métier ; HttpClient décide de la policy ; HttpTransportInterface effectue une requête ; HttpTransportFactory sélectionne le backend disponible.

Annexe A. Aide-mémoire d’une page

J’ai besoin de...	Faire...
Installer	composer require absolutoffice/xcurl:^1.1
Usage normal	new HttpClient(HttpTransportFactory::create()) + send(HttpRequest, RequestOptions)
Framing strict	HttpTransportFactory::createStrict()
Une requête sans policy	$transport->request(...)
Télécharger	RequestOptions(targetPath: $tmp), où $tmp n’existe pas
Body brut	HttpRequest(..., body: $payload); null != ""
Limites	create(maxResponseBytes: ..., maxRequestBytes: ...)
Retries	RetryPolicy ; défaut GET/HEAD, maxRetries=2
Redirects	RedirectPolicy ; défaut maxRedirects=5
Budget global	RequestOptions(deadlineSeconds: ...)
Diagnostiquer backend	HttpTransportFactory::diagnostics()
Échec transport	HttpTransportException: error + transient
Échec client	HttpClientException: error + history
Vérifier runtime	composer check-platform-reqs --no-dev
Régression checkout actuel	composer test + php tests/composer-consumer.php

Ordre de sélection

Mode	1	2	3
create()	cURL	streams	socket
createStrict()	cURL	socket	-

Redirects

301	302	303	307	308
conserve	conserve	GET sauf HEAD ; supprime le body si la méthode change	conserve	conserve